

Course: Individual Software Development Process' 2026

Iteration Report : Iteration 2

***By* Ultrasmooth**

Chosen Irl Challenge : Project A - Lab Data Management



Jirat Kiattrairong	6810545506
Napakhet Namsrioun	6810545727
Panyasiri Aimngern	6810545743
Panisara Niyathirakul	6810545751

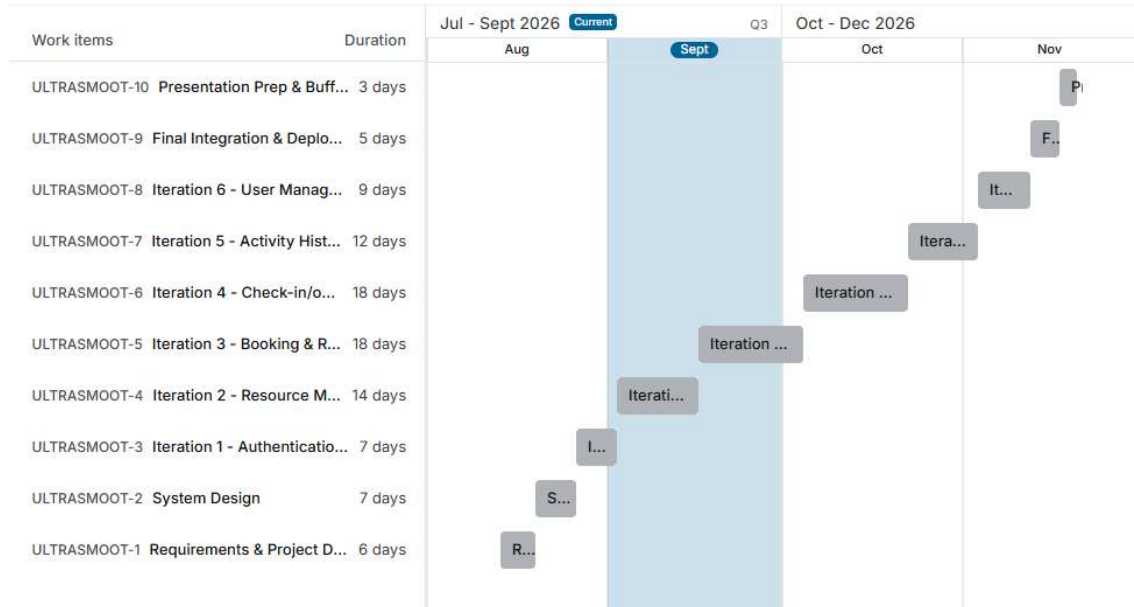
Table of Contents

- 1. Project Snapshot..... 1
 - 1.1 Project-Wide Gantt Chart.....1
 - 1.2 Current Iteration Gantt Chart.....2
- 2. Sprint/Board Status Snapshot.....3
- 3. Scope Addition/Change Log..... 4
- 4. Retrospective..... 4
- 5. Testing & Quality Notes.....5
- 6. Project Risk Register.....6
- 7. Individual Contribution Log..... 8

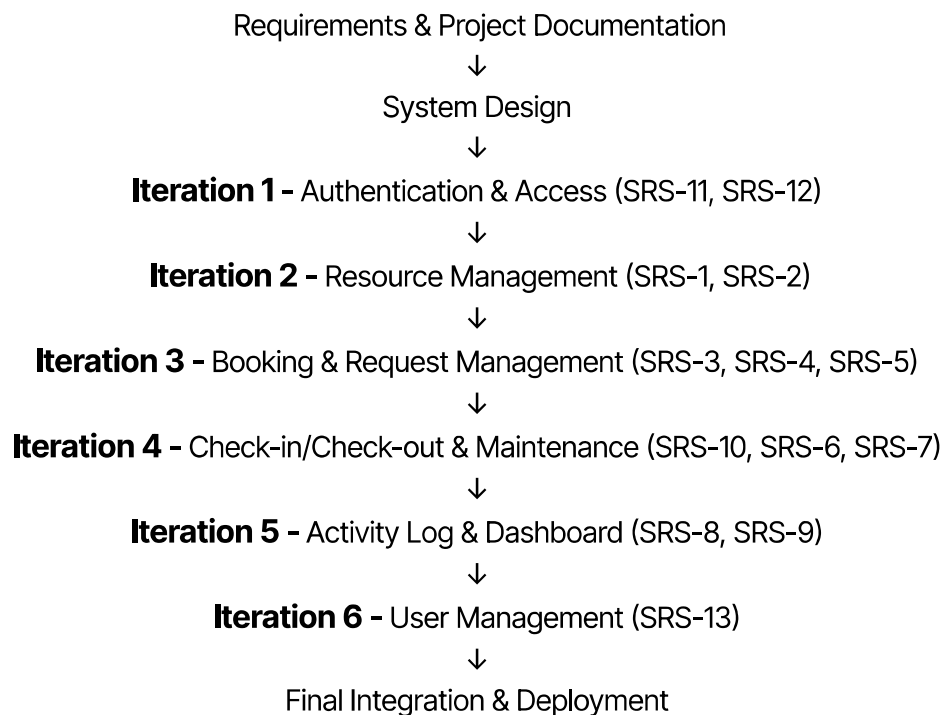
1. Project Snapshot

Project Repository : (WIP)

1.1 Project-Wide Gantt Chart



Critical Path :



Critical Path Duration : 99 calendar days

1.2 Current Iteration Gantt Chart



Critical Path :



Critical Path Duration : 13 calendar days

2. Sprint/Board Status Snapshot

Iteration Goal : The main goal of Iteration 2 was to deliver a working and demo resource inventory feature for the VASE Lab Data Management System. The iteration focused on allowing authenticated users to view and search laboratory resources while the admin role could create, edit and archive resource records.

These features directly support Sub-Goal SG-1 from the SRS, including UC-1 for viewing and searching resources and UC-2 for managing resource records.

During this iteration, the team focused on two user stories :

- **US-1 : View and Search Resources**
- **US-2 : Manage Resource Records**

The implemented features include resource searching, filtering by category, location, and status, resource creation and editing, required-field validation, status validation and resource archiving.

Capacity Declared : The team declared a capacity of 2 user stories for Iteration 2. This was within the maximum capacity of 4 user stories allowed for the iteration.

Both planned user stories, US-1 and US-2, were completed within the iteration. Therefore, the team stayed within the declared capacity and did not need to drop any planned user story.

User Story	Description	Owner	Status	Within Capacity (Y/N)	Notes
US-1	As a Ph.D. Student, Undergraduate Student, Lab Admin, or Professor, I want to view and search resources so I can find what I need.	Panyasiri Aimngern	In Progress	Y	Search by resource name and filtering by category, location and status are implemented. The current resource list uses default name ordering, while the full sorting requirements are not yet implemented. The UI also does not provide a separate availability-specific filter.
US-2	As a Lab Admin, I want to manage resource records so information stays accurate.	Jirat Kiattrairong (Create/Edit) Panisara Niyathirakul (Archive)	Done	Y	Create/Edit is restricted to the admin role. Required-field and status validation prevent invalid saves. Archive changes the record to archived and hides it from the default resource list rather than deleting it.

3. Scope Addition/Change Log

Item added/changed	Reason	What as dropped to compensate (one-in-one-out)
Forgot Password (email-link reset)	The feature was added to improve the existing Iteration 1 login functionality (US-11/SRS-11). It was developed during the Iteration 2 work cycle but is treated as an enhancement to the previous iteration rather than a new Iteration 2 user story.	None. The team continued to complete the two committed Iteration 2 user stories, US-1 and US-2 within the declared capacity.

4. Retrospective

What went well	What didn't go well	Action items for next sprint
Resource management was successfully implemented. The team completed both US-1 and US-2 within the planned iteration. The resource list, search, filtering, create/edit, validation and archive functions were implemented within the iteration.	The resource list does not fully match all SRS-1 requirements yet. The current UI provides search and filters for category, location and status, but the full sorting requirements and a separate availability-specific filter are not yet implemented.	Complete the remaining resource list requirements by implementing the required sorting options and reviewing the availability requirement before the final demonstration.
The team divided the work clearly. US-1 and US-2 were assigned to different members, allowing the List/Search and Create/Edit work to progress without blocking each other.	Some requirements needed closer review before the final demo. The gap between the SRS requirements and the current UI was identified during review rather than during the initial implementation.	Review the SRS before the final demonstration. Check each acceptance criterion against the implemented UI and backend to identify any remaining gaps.
The archive function uses a safer approach. Archiving changes the resource to an archived state instead of permanently deleting the record, while archived resources are excluded from the default active list.	Capacity estimation still has limited historical data. The team completed two user stories, but there is not yet enough historical velocity data to determine whether future capacity estimates are consistently accurate.	Improve future capacity planning. Continue recording actual effort and completion time for each user story to establish a more reliable velocity baseline.

5. Testing & Quality Notes

Note : We did manual and code-level verification on the two stories we built this iteration, US-1 (View and Search Resources) and US-2 (Manage Resource Records).

For US-1, we verified that authenticated users can view and search resources by name and filter by category, location, and status. The current implementation uses name as the default resource ordering, while the full sorting requirements have not yet been implemented.

For US-2, we verified the Create and Edit functions and checked that a resource with missing required fields is rejected with a 400 response and a validation message. We also verified that invalid resource status values are rejected.

For the Archive function, we checked that archiving changes the resource to an archived state without deleting the record and that archived resources are excluded from the default resource list.

During the review, we identified some remaining gaps. The current UI does not provide a separate availability-specific filter, and the full sorting requirements from SRS-1 are not yet implemented. The API also supports the `include_archived` parameter, but the admin UI does not yet provide a clear "Show Archived" option. These findings are currently treated as requirement and UI gaps rather than confirmed defects. Before the final demonstration, the team will review the availability requirement, implement the required sorting options, add the admin-only "Show Archived" option, and perform additional testing after these changes.

6. Project Risk Register

Risk ID	Description	Likelihood (L/M/H)	Impact (L/M/H)	Mitigation/Contingency	Status
R-01	The resource list UI does not yet provide a separate availability-specific filter and the full sorting requirements are not yet implemented. This may prevent the resource list from fully meeting the SRS-1 acceptance criteria during the final demonstration.	M	M	Review the availability requirement and implement the required sorting options before the final demonstration. Test the updated UI against SRS-1.	Open
R-02	Archived resources do not currently have a clear administrator-facing view for review through the UI. Although the API supports the <code>include_archived</code> parameter, the current interface does not expose a "Show Archived" option.	L	M	Add an admin-only "Show Archived" toggle using the existing <code>include_archived</code> parameter. Consider adding an unarchive action if it is required by the final requirements.	Open
R-03	The team's self-declared capacity for Iteration 2 may still be difficult to calibrate because there is limited historical velocity data.	M	M	Continue recording actual completion time and effort per user story. Use the collected data as a baseline for capacity planning in future iterations.	Monitoring

R-04	Some SRS requirements may be missed if the team focuses only on implemented functions without reviewing each acceptance criterion before the final demonstration.	M	M	Perform an SRS-to-implementation review before the final demo and create a checklist covering search, filtering, sorting, create/edit, validation and archive behavior.	Open
------	---	---	---	---	------

7. Individual Contribution Log

Student Name	Tasks owned this sprint	Course concept(s) applied	Evidence	Self-rated effort (1-5)
Napakhet Namsrioun	Forgot Password (Backend) Implemented the backend functionality for the Forgot Password feature. This included password reset token creation and validation, password reset functions, email/reset configuration and the database support required for password reset requests. The feature was treated as an enhancement to the Iteration 1 authentication functionality rather than a new Iteration 2 user story.	Input validation and secure token handling : Applied input validation and secure token handling to the password reset flow. Reset tokens are time-limited and single-use and invalid or expired tokens are rejected before the password can be changed.	8e9ec37 password reset function b056cad password reset table 25af207 password reset and email settings dc869b9 forgot and reset password function	4
Panisara Niyathirakul	Forgot Password (Frontend) Updated index.html for the Forgot Password and Reset Password views and implemented the frontend form and submission handling. Resource Archive : Implemented the resource archive functionality, including the archive action and handling archived resources separately from the active resource list.	Data lifecycle management : The archive function changes the resource to an archived state instead of permanently deleting the record. This keeps the original resource data available while removing archived resources from the normal active view.	35ec99c add forgot password frontend 239418e reset password (email link) 79fbe81 add resource archive dc4d498 add archive resource function	4

Jirat Kiattrairong	Resource: Create & Edit (US-2) : Implemented the backend to create and update resource functions and resource validation. On the frontend, developed the Create/Edit Resource interface, Add Resource button, Resource Form and connection between the frontend form and backend resource functions.	Input validation and CRUD operations : Validation was applied before creating or updating resource records, so incomplete or invalid information cannot be saved. The Create and Edit functions apply CRUD operations to manage resource information while keeping the resource records consistent.	6c55a8f ResourceModel with create/update and validation 1b87679 resource add/edit modal form 3e0e829 create and update endpoints	4
Panyasiri Aimngern	Resource: List & Search (US-1) : On the frontend, implemented the resource list view, search behavior, filter options, default resource ordering, empty-state handling and resource loading.	Filtering and separation of concerns : Filtering by category, location and status helps users narrow the resource list and find the information they need more efficiently. The frontend handles resource presentation and filtering while the backend handles resource retrieval and data processing.	8b080ea add view resources 3c6e741 add the filter option ced5471 add resource_controller.py 00609ab add db.py and schema.sql	4